

Projet Billard 2025

Documentation

Auteurs : Guillaume Saint-Pierre, Louis Gosset, Joseph Belamich

Section : GM

Date : 28 décembre 2025

1. Environnement de développement

- OS + version : macOS 15.0
- Plateforme de test : MacBook Air M1
- Compilateur C : Visual studio Code (1.97.2)
- LabVIEW : 2025 (25.3.2)
- MATLAB : R2025b

2. Organisation des fichiers

Fichier	Rôle	Statut
Fichiers du projet		
Billard2025.vi	VI principal : pipeline complet	Fourni
FindBillardBox.vi	Détection du rectangle intérieur du billard	Fourni
FrameToPosition.vi	Écrit <code>pixmap.bin</code> , lance <code>Pix2Pos</code> , lit <code>pos.txt</code>	Fourni
MatlabScript.vi	Génère le script Matlab et injecte les paramètres	Fourni
RunMatlab.vi	Lance Matlab en ligne de commande et récupère les sorties	Fourni
Pix2Pos.c	Source C : détection des boules et format d'erreur	Fourni
Pix2Pos	L'exécutable obtenu après compilation	Fourni
Fichiers générés pendant l'exécution		
<code>pixmap.bin</code>	Image binaire : W,H + pixels 0x00RRGGBB	Créé
<code>pos.txt</code>	Positions/score : Red/Yellow/White: x, y, score	Créé
<code>AnalyseTx.m</code>	Script Matlab d'analyse généré automatiquement	Créé
<code>ScoreSheetTx.pdf</code>	Score sheet (PDF)	Créé
<code>SummaryTx.txt</code>	Résumé texte compact	Créé

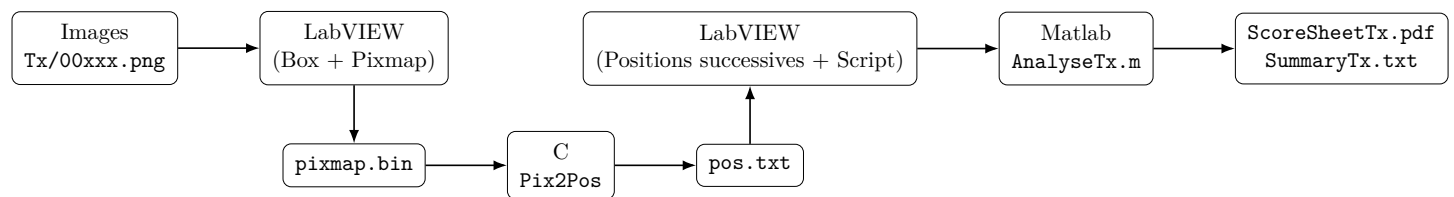
Le VI principal `Billard2025.vi` orchestre l'ensemble du traitement. Le flow de données repose sur des **fichiers intermédiaires** et la récupération des flux `stdout/stderr` lors des appels externes.

Ordre d'exécution

1. LabVIEW charge une image PNG de la séquence.
2. **FindBillardBox.vi** estime la zone intérieure utile (`BillardBox`).
3. **FrameToPosition.vi** convertit la frame en binaire `pixmap.bin`
4. LabVIEW lance `Pix2Pos` via *System Exec* en lui passant **29 paramètres** (rectangle + ranges couleurs + `BallSize`).
5. **Pix2Pos (C)** lit `pixmap.bin` et écrit `pos.txt`.
6. LabVIEW lit `pos.txt` et construit les trajectoires `Xr,Yr,Xy,Yy,Xw,Yw`.

7. **MatlabScript.vi** génère **AnalyseTx.m** en y injectant les positions successives + paramètres (seuils, couleurs de rebonds, etc.).
8. **RunMatlab.vi** exécute Matlab : génération de la figure ScoreSheet, export PDF, création de **SummaryTx.txt**.

3.1 Schéma synthétique



4. Programme C : Pix2Pos

4.1 Principe

- Lit `pixmap.bin` : `width,height (uint32)` puis pixels `0x00RRGGBB`.
- À l'intérieur du rectangle du billard, construit des **masques** (R/Y/W) selon les **plages RGB**.
- Utilise des **images intégrales** pour scorer rapidement chaque fenêtre `BallSize×BallSize`, puis conserve le meilleur score par couleur.

4.2 Gestion des erreurs

Pour que LabVIEW puisse interpréter facilement le résultat :

- **Si tout va bien** : le programme termine avec **exit code = 0**.
- **Si une erreur bloque l'exécution** : le programme écrit un message sur **stderr** sous la forme `ERROR(<code>)` : ... puis termine avec un **exit code négatif**
- **Si c'est juste un avertissement** (ex. trop de pixels, moins de 3 boules détectées) : le programme écrit `WARNING: ...` sur **stderr** mais garde **exit code = 0** pour continuer le pipeline.

4.3 Cas traités (principaux) et exemples de sortie

- **Arguments invalides** : nombre de paramètres incorrect (`argc!=30`) ou valeur non-entière (codes `ERR_PARSE_*`).
- **Fichier d'entrée** : impossible d'ouvrir `pixmap.bin`, fichier trop petit, lecture incomplète.
- **Dimensions** : largeur/hauteur hors bornes `[100..1000]` (avec tentative de correction endianness via `swap32`).
- **Pixels** : pas assez de pixels ⇒ **erreur** ; trop de pixels ⇒ **warning** et pixels excédentaires ignorés.
- **Géométrie** : rectangle du billard invalide ou incohérent avec l'image.
- **Mémoire / sortie** : échec d'allocation (masques, intégrales) ou impossible d'écrire `pos.txt`.
- **Détection partielle** : moins de 3 boules détectées ⇒ **warning** et valeurs manquantes mises à `-1,-1,0`.

Exemples (stderr) :

- `ERROR(-9): Pas assez de pixels dans le fichier` ⇒ `exit=-9`
- `WARNING: Trop de pixels: les pixels excédentaires seront ignorés` ⇒ `exit=0`

5. Matlab : analyse des trajectoires, verdict et génération des sorties

5.1 Entrées / sorties

Le script Matlab est appelé depuis LabVIEW et utilise principalement :

- **Entrées** : `seqName`, `Xr`, `Yr`, `Xy`, `Yy`, `Xw`, `Yw` (même longueur), `MoveDistPx`, `BallBorderDist`, `reboundColorWin`, `reboundColorLoss` (lettre ou `[r g b]` dans `[0..1]`), ainsi que des paramètres de contexte (ex. hauteur image `H=480`, taille boule / seuils selon configuration).
- **Sorties** : génération de la figure *ScoreSheetXX* (export PDF) et écriture d'un résumé *SummaryXX.txt* (format imposé, voir section dédiée).

5.2 Pré-traitement adapté à notre pipeline C → Matlab

Dans `pos.txt`, une boule non détectée est codée `-1,-1,0`. Pour rendre le traitement robuste :

- Les `-1` sont remplacés par `NaN`.
- Passage au repère Matlab (axe vertical inversé grâce à la fonction `flip`) : $y_{mat} = H - y_{img}$
- Les `NaN` sont comblés par la **prochaine valeur valide** (`interp1(...,'next')`), fonction créée dans le script `matlab`) : cela modélise une boule immobile plutôt que de créer un déplacement artificiel.
- Les **outliers** de tracking sont supprimés (`isoutlier` avec médiane glissante) et remplacés par la valeur précédente afin de stabiliser (i) l'affichage, (ii) les distances parcourues, et (iii) la détection de bandes.

5.3 Logique d'analyse : ordre des boules et règle des 3 bandes

Le verdict est déterminé à partir du **début de mouvement** :

- Pour chaque boule, on prend le premier indice où la distance au point initial dépasse `MoveDistPx`. La **1ère boule** est celle avec l'indice minimal (on départage par la fonction `seglen1` si égalité), puis on définit la **2e** et la **3e** par ordre de démarrage.
- Les **bandes** de la 1ère boule sont détectées si elle est à moins de `BallBorderDist` d'un bord du cadre, et on compte celles **strictement entre** le début de mouvement de la 2e et celui de la 3e.
- **Win** si le compteur ≥ 3 , sinon **Loss** (couleur des rebonds selon `reboundColorWin/reboundColorLoss`).

5.4 Gestion d'erreurs

Le script intègre des vérifications explicites pour éviter de produire des sorties incohérentes :

- **Cohérence des trajectoires** : contrôles sur le type, la non-vacuité et l'égalité de longueur (`assertVectorSameLength` + vérification globale).
- **Couleurs invalides** : `validateMatlabColor` refuse toute valeur hors format (lettre autorisée ou RGB dans `[0..1]`), et déclenche une erreur Matlab explicite.
- **Cas limites** : si la 2e ou la 3e boule ne bouge jamais (indice infini), le verdict est automatiquement **Loss** et le script continue avec un affichage cohérent.
- **Écriture Summary** : si le fichier `Summary%s.txt` ne peut pas être ouvert, un **warning** est émis.

6. LabVIEW

6.1 Sous-VI FindBillardBox.vi

Objectif Déterminer automatiquement la **boîte englobante** du cadre *bleu foncé* du billard afin d'obtenir les limites **Xmin, Xmax, Ymin, Ymax**. Ce rectangle ("Billard Box") est ensuite utilisé pour la recherche des boules dans le programme C.

Entrées / sorties

Entrées	Description
Path	Chemin du fichier image (.png)
Dark Blue Frame	Seuils couleur : <code>Rmin/Rmax, Gmin/Gmax, Bmin/Bmax</code>
error in	Chaîne d'erreurs LabVIEW
Sorties	Description
Billard Box	Cluster { <code>Xmin, Xmax, Ymin, Ymax</code> }
error out	Chaîne d'erreurs LabVIEW

Principe

1. **Lecture de l'image** : le fichier PNG est lu, puis converti en données affichables (Picture Control), et enfin en un **tableau 2D** de pixels.
2. **Parcours complet** : deux boucles **For** imbriquées parcourent tous les pixels (**y** pour les lignes, **x** pour les colonnes).
3. **Test “cadre bleu foncé”** : pour chaque pixel, on extrait **R,G,B** et on vérifie :

$$(R \in [R_{\min}, R_{\max}]) \wedge (G \in [G_{\min}, G_{\max}]) \wedge (B \in [B_{\min}, B_{\max}])$$

Les trois tests sont combinés avec une logique **AND** pour produire le booléen **isDarkBlueFrame**.

4. **Mise à jour des extrêmes** (avec **shift registers + Select**) :

— si **isDarkBlueFrame** et **x < Xmin** alors **Xmin := x**

Les comparaisons (**< / >**) sont combinées via **AND** puis appliquées à une fonction **Select** pour choisir entre *la valeur courante* et *la nouvelle valeur*.

5. **Sortie** : en fin de parcours, on regroupe les 4 valeurs dans le cluster **Billard Box**.

Initialisation des registres Les 4 **shift registers** sont initialisés de manière à garantir la première mise à jour : **Xmin** et **Ymin** à une grande valeur (“ $+\infty$ ” approché), **Xmax** et **Ymax** à 0.

6.2 VI FrameToPosition.vi

Objectif À partir d'une frame **.png**, ce VI : (1) lit l'image et l'affiche dans un *Picture Control*, (2) convertit l'image en fichier binaire **Pixmap.bin** au format attendu par le programme C, (3) exécute **Pix2Pos** avec les paramètres (**BillardBox** + seuils **RGB** + **BallDiameter**), (4) lit **Pos.txt** et extrait les positions (**x,y**) et scores des trois boules.

Entrées / sorties

Entrées	Description
PNG Path	Chemin de la frame Tx/00xxx.png
BillardBox	Cluster { Xmin, Xmax, Ymin, Ymax }
Red Range	Seuils { Rmin,Rmax,Gmin,Gmax,Bmin,Bmax }
Yellow Range	Seuils { Rmin,Rmax,Gmin,Gmax,Bmin,Bmax }
White Range	Seuils { Rmin,Rmax,Gmin,Gmax,Bmin,Bmax }
Blue Background Range	Seuils { Rmin,Rmax,Gmin,Gmax,Bmin,Bmax } (param. projet)
BallDiameter	Diamètre (taille carrée) utilisé par Pix2Pos
error in	Chaîne d'erreurs LabVIEW
Sorties	Description
PNG picture	Affichage de la frame (Picture Control)
x/y/score red	Résultat pour la boule rouge
x/y/score yellow	Résultat pour la boule jaune
x/y/score white	Résultat pour la boule blanche
Message C / erreurs	Message renvoyé par l'exécution de Pix2Pos (stderr/stdout)
error out	Chaîne d'erreurs LabVIEW

Principe

1. **Lecture de la frame PNG** : le fichier **PNG Path** est lu et converti en **PNG picture** pour affichage.
2. **Conversion image → Pixmap.bin** :
3. **Préparation de la ligne de commande Pix2Pos** : les paramètres sont ordonnés et concaténés en une chaîne de 29 arguments pour la ligne de commande du C.
4. **Exécution du programme C** : appel de **./Pix2Pos** via un nœud d'exécution système. Le message de retour (stdout/stderr) est récupéré dans **Message C / erreurs**.
5. **Lecture et parsing de Pos.txt** : **Pos.txt** est ouvert puis converti en chaîne. L'extraction des valeurs se fait avec un **Scan From String**.

6.3 VI MatlabScript.vi

Objectif Ce VI construit le script Matlab d'analyse (**AnalyseTx.m**) à partir des données extraites précédemment (**Pos.txt**) et des paramètres de configuration LabVIEW. Il génère un fichier texte contenant les instructions Matlab prêtes à exécution, en intégrant automatiquement les paramètres. Le fichier Matlab est ensuite enregistré dans le répertoire de travail et sera exécuté par **RunMatlab.vi**.

Entrées / Sorties

Entrées	Description
Xr, Yr, Xy, Yy, Xw, Yw	Coordonnées des trajectoires des boules rouge, jaune, blanche.
reboundColorWin	Couleur des rebonds en cas de réussite.
reboundColorLoss	Couleur des rebonds en cas d'échec.
BallBorderDist	Distance au bord du billard pour détecter les rebonds.
MoveDistPx	Seuil de déplacement (px) pour détecter le mouvement d'une boule.
BallminScore	Score minimal accepté pour valider une détection C.
Path MatlabScript	Chemin complet du fichier AnalyseTx.m à créer.
error in	Chaîne d'erreurs LabVIEW en entrée.
Sorties	Description
Script Path	Chemin complet du fichier Matlab généré.
error out	Chaîne d'erreurs LabVIEW en sortie.

Principe

1. **Préparation des données** : Lecture des tableaux de coordonnées **Xr...Yw** et conversion en chaînes Matlab (**sprintf**) au format **[x1 x2 x3 ...]**.
2. **Génération du texte** : Concatenation ligne par ligne du script Matlab.
3. **Écriture du fichier** : Le texte final est sauvegardé dans un fichier **Analyse<seqName>.m** via la fonction **Write to Text File**.

6.4 Gestion des erreurs Labview

Propagation du *error cluster* Nous avons tout d'abord relié le fil d'erreur (*error cluster*) à l'ensemble des sous-VI et des étapes principales, ce qui garantit la propagation automatique d'une erreur détectée à n'importe quel niveau du diagramme.

Arrêt des traitements en cas d'erreur (machines d'état) Pour éviter d'exécuter des tâches longues ou inutiles lorsqu'une erreur est déjà présente, nous avons intégré des machines d'état dont les transitions sont conditionnées par l'inverse du statut d'erreur (**NOT error.status**).

Intégration des erreurs du code C (Pix2Pos) Le programme C renvoie un message explicite via **stderr**. Nous récupérons ce message et, en cas d'échec, nous l'injectons dans le champ **error.source** en utilisant une fonction **Select** (erreur si différent de 0).

7. Commentaires

Convention de repère Nous avons fait le choix d'un repère avec *x* vers la droite et *y* vers le haut. Nous corrigeons simplement cet ordre lors de la construction de la ligne de commande pour **Pix2Pos**. Ce choix est purement une convention interne et n'a aucun impact sur les résultats du projet.

Matlab Score Sheet Si l'option **Matlab Score Sheet** est désactivée, le script Matlab n'est pas généré et Matlab n'est pas lancé. Dans ce cas, si aucune erreur n'est détectée, le programme écrit volontairement un **Summary** vide afin d'éviter de réutiliser par erreur le **Summary** généré lors d'une exécution précédente (cf. question #327).